Projet IA Classification Des Panneaux De Signalisation

April 12, 2026

Réalisé par :

- DEGNY GILLES ALFRED
- KOUTOUA CHRISTOPHER

Niveau : MASTER 2 MIAGE

1 Introduction

Ceci est un rapport Technique pour un Système de Reconnaissance Automatique de Pannequx de Signalisation basé sur l'Intelligence Artificielle. Ce sujet touche aux domaines de la Computer Vision et du Deep Learning. Le point marquant est la mise en place du Pipeline de traitement d'images de bout-en-bout.

1.1 OBJECTIF DU PROJET

L'objectif principal de ce travail est double :

- construire un modèle CNN
- classifier automatiquement les panneaux routiers

Autrement dit, on veut entraîner un programme à regarder une image et à dire à quelle catégorie de panneau elle appartient.

1.2 PRÉSENTATION DU JEU DE DONNÉES

Pour réaliser ce projet, on utilise le dataset GTSRB (German Traffic Sign Recognition Benchmark), proposé sur Kaggle. Ce jeu de données contient plus de 50000 images de panneaux de signalisation, associées à leurs étiquettes. Chaque image appartient donc à une classe précise, ce qui permet d'entraîner puis d'évaluer le modèle.

1.3 QU'EST-CE QU'UN CNN ?

Un CNN (Convolutional Neural Network) est un type de réseau de neurones spécialement conçu pour travailler sur les images. Son rôle est d'apprendre automatiquement les éléments visuels importants d'une image, par exemple :

- les contours
- les formes
- les couleurs
- les symboles

Dans notre cas, le CNN doit apprendre à reconnaître les caractéristiques visuelles propres à chaque panneau de signalisation afin de les classer correctement. En général, un CNN contient des couches de convolution, de max pooling, puis des couches comme flatten et dense pour produire la prédiction finale. Cette logique correspond à la méthodologie demandée dans le sujet.

2 Initialisation du Pipeline technique

Dans cette section, nous configurons l'environnement d'exécution, les bibliothèques de calcul intensif (TensorFlow/Keras) et les paramètres globaux du projet.

```python
# ==========================================================
# 1. IMPORTS
# ==========================================================

import tensorflow as tf
import pandas as pd
import numpy as np
import os
import random
import matplotlib.pyplot as plt
import seaborn as sns
import warnings

from PIL import Image
from pathlib import Path

from sklearn.model_selection import train_test_split
from sklearn.metrics import confusion_matrix, classification_report

warnings.filterwarnings('ignore')

print(f"TensorFlow version : {tf.__version__}")
print(f"Pandas version     : {pd.__version__}")
print("Environnement prêt !")
```

```
TensorFlow version : 2.21.0
Pandas version     : 3.0.2
Environnement prêt !
```

```python
# ============================================================
# 2. PARAMETRES GENERAUX
# ============================================================

IMAGE_SIZE = (32, 32)
BATCH_SIZE = 32
EPOCHS = 15
SEED = 42
N_CLASSES = 43

# Pour avoir des résultats reproductibles
np.random.seed(SEED)
random.seed(SEED)
tf.random.set_seed(SEED)

# Pour l'affichage
sns.set_style("whitegrid")
plt.rcParams["figure.figsize"] = (10, 5)
```

2.1 Acquisition et Intégration des Données

```python
# ============================================================
# 3. CHEMINS
#    Ici, le notebook Init.ipynb est au même niveau que GTSRB
# ============================================================

base_path = Path("GTSRB")

train_path = base_path / "Train"
test_path = base_path / "Test"
meta_path = base_path / "Meta"

train_csv_path = base_path / "Train.csv"
csv_path = base_path / "Test.csv"
meta_csv_path = base_path / "Meta.csv"

print("Train path :", train_path)
print("Test path  :", test_path)
print("Meta path  :", meta_path)

print("Train.csv trouvé :", train_csv_path.exists())
print("Test.csv trouvé  :", csv_path.exists())
print("Meta.csv trouvé  :", meta_csv_path.exists())
```

```
Train path : GTSRB/Train
Test path  : GTSRB/Test
Meta path  : GTSRB/Meta
```

```
Train.csv trouvé : True
Test.csv trouvé  : True
Meta.csv trouvé  : True
```

```python
# ==========================================================
# 4. CHARGEMENT DES CSV
# ==========================================================

data = pd.read_csv(train_csv_path)
test_df = pd.read_csv(csv_path)
meta_df = pd.read_csv(meta_csv_path)

print("Taille data (train officiel) :", data.shape)
print("Taille test_df (test officiel) :", test_df.shape)
print("Taille meta_df :", meta_df.shape)

display(data.head())
display(test_df.head())
```

```
Taille data (train officiel) : (39209, 8)
Taille test_df (test officiel) : (12630, 8)
Taille meta_df : (43, 5)
```

	Width	Height	Roi.X1	Roi.Y1	Roi.X2	Roi.Y2	ClassId	\
0	27	26	5	5	22	20	20	
1	28	27	5	6	23	22	20	
2	29	26	6	5	24	21	20	
3	28	27	5	6	23	22	20	
4	28	26	5	5	23	21	20	

	Path
0	Train/20/00020_00000_00000.png
1	Train/20/00020_00000_00001.png
2	Train/20/00020_00000_00002.png
3	Train/20/00020_00000_00003.png
4	Train/20/00020_00000_00004.png

	Width	Height	Roi.X1	Roi.Y1	Roi.X2	Roi.Y2	ClassId	Path
0	53	54	6	5	48	49	16	Test/00000.png
1	42	45	5	5	36	40	1	Test/00001.png
2	48	52	6	6	43	47	38	Test/00002.png
3	27	29	5	5	22	24	33	Test/00003.png
4	60	57	5	5	55	52	11	Test/00004.png

```python
# ============================================================
# 5. CHEMINS COMPLETS DES IMAGES
#    Très important : on utilise les vrais labels ClassId des CSV
# ============================================================

data["full_path"] = data["Path"].apply(lambda x: str(base_path / x))
test_df["full_path"] = test_df["Path"].apply(lambda x: str(base_path / x))
meta_df["full_path"] = meta_df["Path"].apply(lambda x: str(base_path / x))

# Labels du train officiel
labels = data["ClassId"].values

display(data[["Path", "full_path", "ClassId"]].head())
display(test_df[["Path", "full_path", "ClassId"]].head())
```

```
                            Path                              full_path  \
0  Train/20/00020_00000_00000.png  GTSRB/Train/20/00020_00000_00000.png
1  Train/20/00020_00000_00001.png  GTSRB/Train/20/00020_00000_00001.png
2  Train/20/00020_00000_00002.png  GTSRB/Train/20/00020_00000_00002.png
3  Train/20/00020_00000_00003.png  GTSRB/Train/20/00020_00000_00003.png
4  Train/20/00020_00000_00004.png  GTSRB/Train/20/00020_00000_00004.png

   ClassId
0       20
1       20
2       20
3       20
4       20

             Path             full_path  ClassId
0  Test/00000.png  GTSRB/Test/00000.png       16
1  Test/00001.png  GTSRB/Test/00001.png        1
2  Test/00002.png  GTSRB/Test/00002.png       38
3  Test/00003.png  GTSRB/Test/00003.png       33
4  Test/00004.png  GTSRB/Test/00004.png       11
```

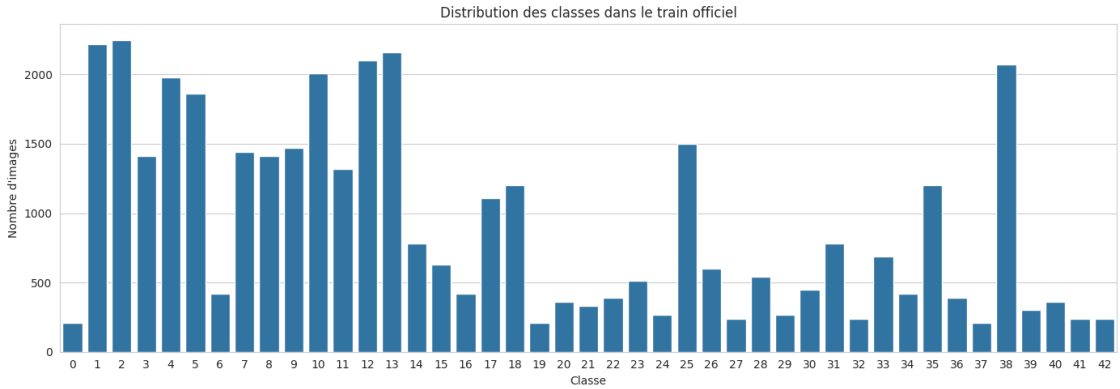
3 Analyse Exploratoire des Données (EDA)

L'objectif est de comprendre la distribution des 43 classes de panneaux et d'identifier d'éventuels déséquilibres (imbalance) qui pourraient biaiser l'apprentissage.

```python
# ==========================================================
# 6. VERIFICATION RAPIDE
# ==========================================================

print("Nombre de classes dans le train :", data["ClassId"].nunique())
print("Classes présentes :", sorted(data["ClassId"].unique()))
```

```
Nombre de classes dans le train : 43
Classes présentes : [np.int64(0), np.int64(1), np.int64(2), np.int64(3),
np.int64(4), np.int64(5), np.int64(6), np.int64(7), np.int64(8), np.int64(9),
np.int64(10), np.int64(11), np.int64(12), np.int64(13), np.int64(14),
np.int64(15), np.int64(16), np.int64(17), np.int64(18), np.int64(19),
np.int64(20), np.int64(21), np.int64(22), np.int64(23), np.int64(24),
np.int64(25), np.int64(26), np.int64(27), np.int64(28), np.int64(29),
np.int64(30), np.int64(31), np.int64(32), np.int64(33), np.int64(34),
np.int64(35), np.int64(36), np.int64(37), np.int64(38), np.int64(39),
np.int64(40), np.int64(41), np.int64(42)]
```

```python
# ==========================================================
# 7. DISTRIBUTION DES CLASSES
# ==========================================================

distribution = data["ClassId"].value_counts().sort_index()

plt.figure(figsize=(14, 5))
sns.barplot(x=distribution.index, y=distribution.values)
plt.title("Distribution des classes dans le train officiel")
plt.xlabel("Classe")
plt.ylabel("Nombre d'images")
plt.tight_layout()
plt.show()
```



```python
# ============================================================
# 8. AFFICHAGE DE QUELQUES IMAGES
#    Cela respecte la partie exploration / chargement avec PIL
# ============================================================

sample_df = data.sample(n=9, random_state=SEED).reset_index(drop=True)

plt.figure(figsize=(12, 10))

for i in range(9):
    img_path = sample_df.loc[i, "full_path"]
    real_class = int(sample_df.loc[i, "ClassId"])

    img = Image.open(img_path).convert("RGB")

    plt.subplot(3, 3, i + 1)
    plt.imshow(img)
    plt.title(f"Classe : {real_class}")
    plt.axis("off")

plt.tight_layout()
plt.show()
```

Classe : 26
Classe : 14
STOP
Classe : 13
Classe : 10
Classe : 15
Classe : 12
Classe : 25
Classe : 2
50
Classe : 11

4 Feature Engineering & Prétraitement

```
# ==========================================================
# 9. DIVISION DU TRAIN OFFICIEL EN TRAIN + VALIDATION
#    Le test officiel reste intact
# ==========================================================

X_train, X_val = train_test_split(
    data,
    test_size=0.20,
    random_state=SEED,
    stratify=data["ClassId"]
)

# Le test officiel reste séparé
X_test = test_df.copy()

# Labels correspondants
y_train = X_train["ClassId"].values
y_val = X_val["ClassId"].values
y_test = X_test["ClassId"].values

# Réindexation propre
X_train = X_train.reset_index(drop=True)
X_val = X_val.reset_index(drop=True)
X_test = X_test.reset_index(drop=True)

print("Taille X_train :", X_train.shape)
print("Taille X_val   :", X_val.shape)
print("Taille X_test  :", X_test.shape)
```

```
Taille X_train : (31367, 9)
Taille X_val   : (7842, 9)
Taille X_test  : (12630, 9)
```

```python
# ==========================================================
# 10. GENERATEUR DE DONNEES
#     Il lit les images par lots pour éviter de tout charger
#     en mémoire en une seule fois
# ==========================================================

class TrafficSignGenerator(tf.keras.utils.Sequence):
    def __init__(self, dataframe, batch_size=BATCH_SIZE, image_size=IMAGE_SIZE,␣
 ↪shuffle=True):
        self.dataframe = dataframe.reset_index(drop=True).copy()
        self.batch_size = batch_size
        self.image_size = image_size
        self.shuffle = shuffle
        self.indices = np.arange(len(self.dataframe))
        self.on_epoch_end()

    def __len__(self):
        # Nombre de lots par époque
        return int(np.ceil(len(self.dataframe) / self.batch_size))

    def __getitem__(self, index):
        # Sélection des indices du lot
        start = index * self.batch_size
        end = min((index + 1) * self.batch_size, len(self.dataframe))
        batch_indices = self.indices[start:end]

        batch_df = self.dataframe.iloc[batch_indices]

        # Tableaux du lot
        batch_images = np.zeros(
            (len(batch_df), self.image_size[0], self.image_size[1], 3),
            dtype=np.float32
        )
        batch_labels = batch_df["ClassId"].astype(np.int32).values

        # Lecture image par image
        for i, path in enumerate(batch_df["full_path"]):
            img = Image.open(path).convert("RGB")
            img = img.resize(self.image_size)

            img_array = np.array(img, dtype=np.float32) / 255.0
            batch_images[i] = img_array

        return batch_images, batch_labels

    def on_epoch_end(self):
        # Mélange à la fin de chaque époque
```

```
        if self.shuffle:
            np.random.shuffle(self.indices)
```

```python
# ============================================================
# 11. CREATION DES GENERATEURS
# ============================================================

train_generator = TrafficSignGenerator(
    X_train,
    batch_size=BATCH_SIZE,
    image_size=IMAGE_SIZE,
    shuffle=True
)

val_generator = TrafficSignGenerator(
    X_val,
    batch_size=BATCH_SIZE,
    image_size=IMAGE_SIZE,
    shuffle=False
)

test_generator = TrafficSignGenerator(
    X_test,
    batch_size=BATCH_SIZE,
    image_size=IMAGE_SIZE,
    shuffle=False
)

print("Nombre de lots train :", len(train_generator))
print("Nombre de lots val   :", len(val_generator))
print("Nombre de lots test  :", len(test_generator))
```

```
Nombre de lots train : 981
Nombre de lots val   : 246
Nombre de lots test  : 395
```

```python
# ============================================================
# 12. VERIFICATION D'UN LOT
# ============================================================

batch_images, batch_labels = train_generator[0]

print("Forme batch_images :", batch_images.shape)
print("Forme batch_labels :", batch_labels.shape)
print("Quelques labels    :", batch_labels[:10])
```

```
Forme batch_images : (32, 32, 32, 3)
Forme batch_labels : (32,)
Quelques labels    : [21 22 20  8 24  4  3  9  5  2]
```

5 Conception de l'Architecture Neuronale (CNN)

Nous implémentons une architecture de réseau de neurones convolutifs (CNN) optimisée pour la classification d'images à basse résolution (32x32 pixels).

```
# ==========================================================
# 13. CONSTRUCTION DU MODELE CNN
#     On utilise bien TensorFlow / Keras
# ==========================================================

from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Input, Conv2D, MaxPooling2D, Flatten,␣
 ↪Dense, Dropout

model = Sequential([
    Input(shape=(IMAGE_SIZE[0], IMAGE_SIZE[1], 3)),

    # Bloc 1
    Conv2D(32, (3, 3), activation="relu", padding="same"),
    MaxPooling2D((2, 2)),

    # Bloc 2
    Conv2D(64, (3, 3), activation="relu", padding="same"),
    MaxPooling2D((2, 2)),

    # Bloc 3
    Conv2D(128, (3, 3), activation="relu", padding="same"),
    MaxPooling2D((2, 2)),

    # Passage vers les couches denses
    Flatten(),
    Dense(128, activation="relu"),
    Dropout(0.4),

    # Couche de sortie
    Dense(N_CLASSES, activation="softmax")
])

model.summary()
```

```
E0000 00:00:1775975477.923872   52438 cuda_platform.cc:52] failed call to
cuInit: INTERNAL: CUDA error: Failed call to cuInit: UNKNOWN ERROR (303)
```

Model: "sequential"

Layer (type)	Output Shape	Param #

conv2d (Conv2D)	(None, 32, 32, 32)	896
max_pooling2d (MaxPooling2D)	(None, 16, 16, 32)	0
conv2d_1 (Conv2D)	(None, 16, 16, 64)	18,496
max_pooling2d_1 (MaxPooling2D)	(None, 8, 8, 64)	0
conv2d_2 (Conv2D)	(None, 8, 8, 128)	73,856
max_pooling2d_2 (MaxPooling2D)	(None, 4, 4, 128)	0
flatten (Flatten)	(None, 2048)	0
dense (Dense)	(None, 128)	262,272
dropout (Dropout)	(None, 128)	0
dense_1 (Dense)	(None, 43)	5,547

Total params: 361,067 (1.38 MB)

Trainable params: 361,067 (1.38 MB)

Non-trainable params: 0 (0.00 B)

6 Entraînement et Monitoring des Performances

```
# ==========================================================
# 14. COMPILATION DU MODELE
#     Adam + sparse_categorical_crossentropy
#     car les labels sont des entiers (0 à 42)
# ==========================================================

model.compile(
    optimizer="adam",
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"]
)
```

```
# ==========================================================
# 15. CALLBACKS
#     EarlyStopping : arrête si la validation ne progresse plus
#     ReduceLROnPlateau : baisse le learning rate si besoin
# ==========================================================

early_stopping = tf.keras.callbacks.EarlyStopping(
    monitor="val_loss",
    patience=3,
    restore_best_weights=True,
    verbose=1
)

reduce_lr = tf.keras.callbacks.ReduceLROnPlateau(
    monitor="val_loss",
    factor=0.5,
    patience=2,
    verbose=1
)
```

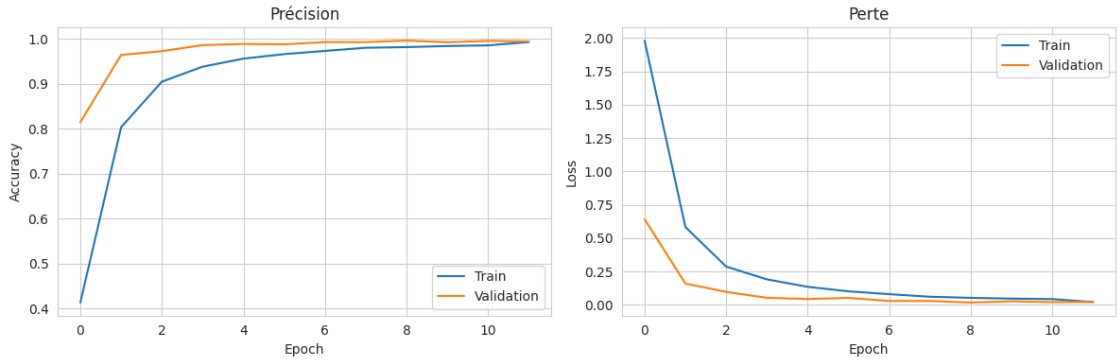
```
# ==========================================================
# 16. ENTRAINEMENT
# ==========================================================

history = model.fit(
    train_generator,
    validation_data=val_generator,
    epochs=EPOCHS,
    callbacks=[early_stopping, reduce_lr],
    verbose=1
)
```

```
Epoch 1/15
```

```
I0000 00:00:1775975543.759155   52438 generator_dataset_op.cc:213] Memory patch
applied: M_TRIM_THRESHOLD=128 kb was set.

  1/981              38:53 2s/step - accuracy:
0.0000e+00 - loss: 3.7644

W0000 00:00:1775975546.056053   64723 cpu_allocator_impl.cc:82] Allocation of
21528576 exceeds 10% of free system memory.
W0000 00:00:1775975546.056171   64724 cpu_allocator_impl.cc:82] Allocation of
21528576 exceeds 10% of free system memory.

  2/981              27:47 2s/step - accuracy:
0.0078 - loss: 3.7646

W0000 00:00:1775975547.752381   64721 cpu_allocator_impl.cc:82] Allocation of
21528576 exceeds 10% of free system memory.
W0000 00:00:1775975547.752475   64724 cpu_allocator_impl.cc:82] Allocation of
21528576 exceeds 10% of free system memory.

  3/981              31:51 2s/step - accuracy:
0.0087 - loss: 3.7626

W0000 00:00:1775975549.975870   64724 cpu_allocator_impl.cc:82] Allocation of
21528576 exceeds 10% of free system memory.

981/981              446s 453ms/step -
accuracy: 0.4139 - loss: 1.9810 - val_accuracy: 0.8151 - val_loss: 0.6422 -
learning_rate: 0.0010
Epoch 2/15
981/981              77s 79ms/step -
accuracy: 0.8038 - loss: 0.5841 - val_accuracy: 0.9647 - val_loss: 0.1601 -
learning_rate: 0.0010
Epoch 3/15
981/981              80s 82ms/step -
accuracy: 0.9052 - loss: 0.2873 - val_accuracy: 0.9730 - val_loss: 0.0974 -
learning_rate: 0.0010
Epoch 4/15
981/981              81s 81ms/step -
accuracy: 0.9385 - loss: 0.1914 - val_accuracy: 0.9865 - val_loss: 0.0535 -
learning_rate: 0.0010
Epoch 5/15
981/981              80s 82ms/step -
accuracy: 0.9563 - loss: 0.1354 - val_accuracy: 0.9890 - val_loss: 0.0438 -
learning_rate: 0.0010
Epoch 6/15
981/981              85s 85ms/step -
accuracy: 0.9664 - loss: 0.1017 - val_accuracy: 0.9881 - val_loss: 0.0525 -
learning_rate: 0.0010
Epoch 7/15
981/981              79s 80ms/step -
accuracy: 0.9735 - loss: 0.0803 - val_accuracy: 0.9931 - val_loss: 0.0292 -
```

```
learning_rate: 0.0010
Epoch 8/15
981/981            80s 82ms/step -
accuracy: 0.9806 - loss: 0.0612 - val_accuracy: 0.9932 - val_loss: 0.0293 -
learning_rate: 0.0010
Epoch 9/15
981/981            82s 83ms/step -
accuracy: 0.9820 - loss: 0.0528 - val_accuracy: 0.9963 - val_loss: 0.0179 -
learning_rate: 0.0010
Epoch 10/15
981/981            80s 82ms/step -
accuracy: 0.9844 - loss: 0.0471 - val_accuracy: 0.9929 - val_loss: 0.0269 -
learning_rate: 0.0010
Epoch 11/15
981/981            0s 76ms/step -
accuracy: 0.9853 - loss: 0.0420
Epoch 11: ReduceLROnPlateau reducing learning rate to 0.0005000000237487257.
981/981            81s 82ms/step -
accuracy: 0.9859 - loss: 0.0435 - val_accuracy: 0.9957 - val_loss: 0.0203 -
learning_rate: 0.0010
Epoch 12/15
981/981            81s 83ms/step -
accuracy: 0.9932 - loss: 0.0211 - val_accuracy: 0.9948 - val_loss: 0.0223 -
learning_rate: 5.0000e-04
Epoch 12: early stopping
Restoring model weights from the end of the best epoch: 9.
```

```python
# ============================================================
# 17. COURBES D'APPRENTISSAGE
# ============================================================

plt.figure(figsize=(12, 4))

# Accuracy
plt.subplot(1, 2, 1)
plt.plot(history.history["accuracy"], label="Train")
plt.plot(history.history["val_accuracy"], label="Validation")
plt.title("Précision")
plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.legend()

# Loss
plt.subplot(1, 2, 2)
plt.plot(history.history["loss"], label="Train")
plt.plot(history.history["val_loss"], label="Validation")
plt.title("Perte")
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.legend()

plt.tight_layout()
plt.show()
```



7 Évaluation Critique et Métriques de Performance

```
# ==========================================================
# 18. EVALUATION SUR L'ENSEMBLE DE VALIDATION
# ==========================================================

val_loss, val_acc = model.evaluate(val_generator, verbose=1)

print(f"Loss validation     : {val_loss:.4f}")
print(f"Accuracy validation : {val_acc:.4f}")
```

```
  1/246              19s 79ms/step - accuracy:
1.0000 - loss: 0.0016

246/246              7s 28ms/step -
accuracy: 0.9963 - loss: 0.0179
Loss validation     : 0.0179
Accuracy validation : 0.9963
```

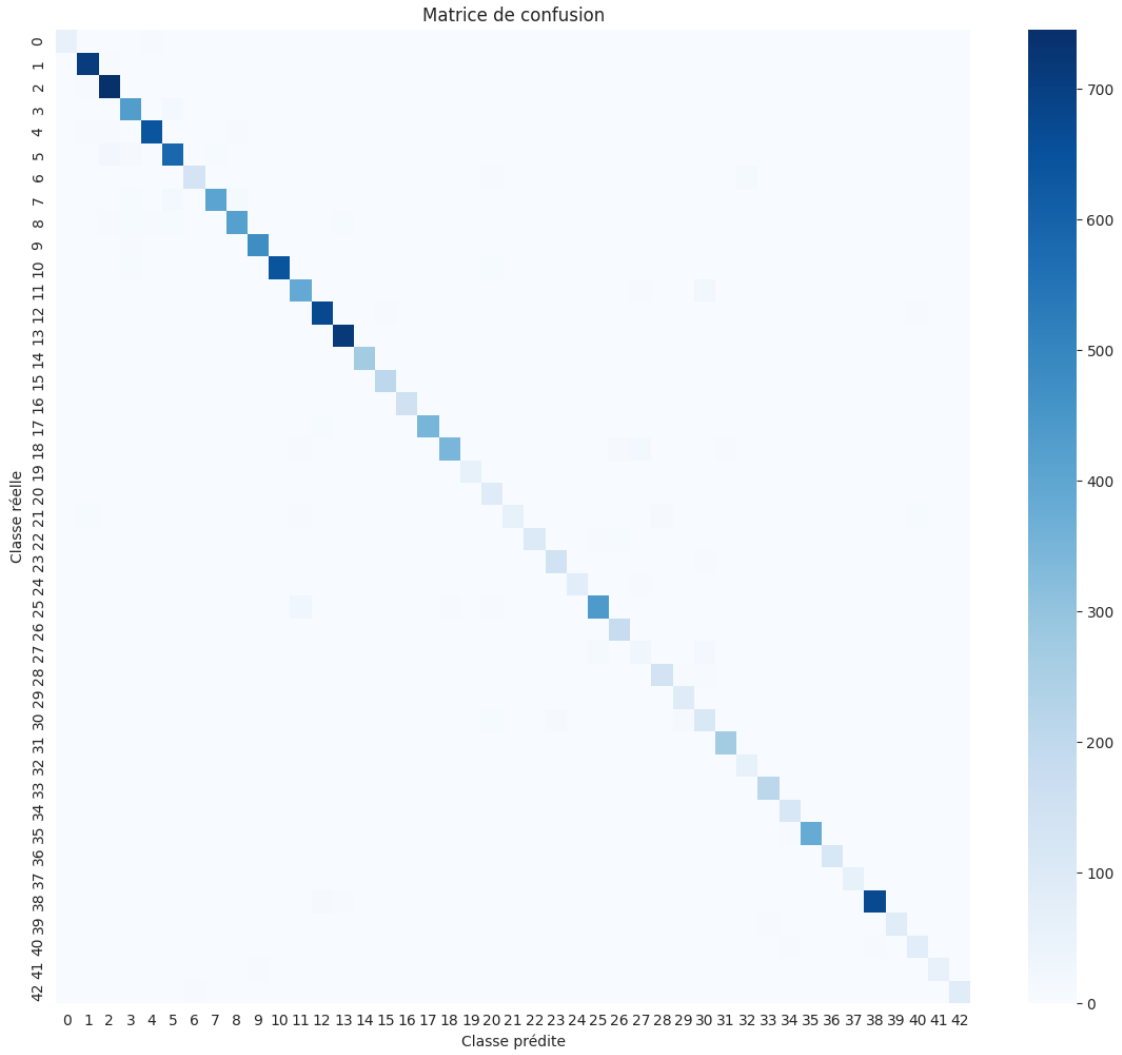
```
# ==========================================================
# 19. EVALUATION SUR LE TEST OFFICIEL
# ==========================================================

loss, acc = model.evaluate(test_generator, verbose=1)

print(f"Loss test     : {loss:.4f}")
print(f"Accuracy test : {acc:.4f}")
```

```
395/395              14s 36ms/step -
accuracy: 0.9568 - loss: 0.2272
Loss test     : 0.2272
Accuracy test : 0.9568
```

```
# ==========================================================
# 20. PREDICTIONS SUR LE TEST OFFICIEL
# ==========================================================

y_pred_proba = model.predict(test_generator, verbose=1)
y_pred = np.argmax(y_pred_proba, axis=1)

print("Nombre de vraies étiquettes :", len(y_test))
print("Nombre de prédictions       :", len(y_pred))
```

```
395/395              12s 29ms/step
Nombre de vraies étiquettes : 12630
Nombre de prédictions       : 12630
```

```python
# ==========================================================
# 21. MATRICE DE CONFUSION
# ==========================================================

cm = confusion_matrix(y_test, y_pred)

plt.figure(figsize=(14, 12))
sns.heatmap(cm, annot=False, cmap="Blues")
plt.title("Matrice de confusion")
plt.xlabel("Classe prédite")
plt.ylabel("Classe réelle")
plt.show()
```



```python
# ============================================================
# 22. RAPPORT DE CLASSIFICATION
# ============================================================

print(classification_report(y_test, y_pred, digits=4))
```

```
              precision    recall  f1-score   support

           0     0.9825    0.9333    0.9573        60
           1     0.9753    0.9861    0.9807       720
           2     0.9527    0.9933    0.9726       750
           3     0.9206    0.9533    0.9367       450
           4     0.9831    0.9697    0.9764       660
           5     0.9184    0.9286    0.9234       630
           6     0.9776    0.8733    0.9225       150
           7     0.9693    0.9111    0.9393       450
           8     0.9567    0.9333    0.9449       450
           9     0.9835    0.9917    0.9876       480
          10     0.9954    0.9742    0.9847       660
          11     0.9065    0.9238    0.9151       420
          12     0.9671    0.9797    0.9734       690
          13     0.9809    0.9972    0.9890       720
          14     0.9963    1.0000    0.9982       270
          15     0.9812    0.9952    0.9882       210
          16     0.9868    0.9933    0.9900       150
          17     1.0000    0.9750    0.9873       360
          18     0.9692    0.8872    0.9264       390
          19     0.9833    0.9833    0.9833        60
          20     0.7807    0.9889    0.8725        90
          21     0.9077    0.6556    0.7613        90
          22     0.9811    0.8667    0.9204       120
          23     0.9346    0.9533    0.9439       150
          24     0.9767    0.9333    0.9545        90
          25     0.9543    0.9146    0.9340       480
          26     0.9086    0.9944    0.9496       180
          27     0.5263    0.5000    0.5128        60
          28     0.9085    0.9267    0.9175       150
          29     0.8108    1.0000    0.8955        90
          30     0.6890    0.7533    0.7197       150
          31     0.9853    0.9963    0.9908       270
          32     0.8108    1.0000    0.8955        60
          33     0.9859    1.0000    0.9929       210
          34     0.9520    0.9917    0.9714       120
          35     1.0000    0.9846    0.9922       390
          36     0.9832    0.9750    0.9791       120
          37     0.9836    1.0000    0.9917        60
          38     0.9897    0.9768    0.9832       690
          39     1.0000    0.9444    0.9714        90
```

40	0.8646	0.9222	0.8925	90
41	1.0000	0.9333	0.9655	60
42	0.9886	0.9667	0.9775	90
accuracy			0.9568	12630
macro avg	0.9374	0.9386	0.9363	12630
weighted avg	0.9581	0.9568	0.9567	12630

8 Inférence et Tests Unitaires sur Données Réelles

```
# ============================================================
# 23. FONCTION DE PREDICTION SUR UNE IMAGE
# ============================================================

def predire_une_image(img_path):
    # Chargement de l'image
    img = Image.open(img_path).convert("RGB")
    img_resized = img.resize(IMAGE_SIZE)

    # Prétraitement
    img_array = np.array(img_resized, dtype=np.float32) / 255.0
    img_array = img_array.reshape(1, IMAGE_SIZE[0], IMAGE_SIZE[1], 3)

    # Prédiction
    prediction = model.predict(img_array, verbose=0)
    predicted_class = int(np.argmax(prediction))
    confidence = float(np.max(prediction))

    return img, predicted_class, confidence
```

```
# ============================================================
# 24. TEST SUR QUELQUES IMAGES ALEATOIRES DU TEST
# ============================================================

nombre_images = min(12, len(test_df))

# Sans random_state : images différentes à chaque exécution
echantillon_test = test_df.sample(n=nombre_images).reset_index(drop=True)

plt.figure(figsize=(15, 12))

for i in range(len(echantillon_test)):
    random_row = echantillon_test.loc[i]

    img_path = random_row["full_path"]
    real_class = int(random_row["ClassId"])

    img, predicted_class, confidence = predire_une_image(img_path)

    plt.subplot(3, 4, i + 1)
    plt.imshow(img)
    plt.title(
        f"Réelle : {real_class}\nPrédite : {predicted_class}\nConf. :␣
↪{confidence:.3f}",
        fontsize=10
```

```
    )
    plt.axis("off")

plt.tight_layout()
plt.show()
```

```python
# ==========================================================
# 25. TEST SUR UNE SEULE IMAGE ALEATOIRE
# ==========================================================

random_row = test_df.sample(1).iloc[0]

img_path = random_row["full_path"]
real_class = int(random_row["ClassId"])

img, predicted_class, confidence = predire_une_image(img_path)

plt.figure(figsize=(5, 5))
plt.imshow(img)
plt.title(
    f"Classe réelle : {real_class}\n"
    f"Classe prédite : {predicted_class}\n"
    f"Confiance : {confidence:.4f}"
)
plt.axis("off")
plt.show()
```